Intel® Intrinsics Guide

Updated 07/12/2024 Version 3.6.9

Instruction Set

- ☐ MMX
- ☐ SSE family
- ☐ AVX family
- ☐ AVX-512 family
- ☐ AMX family
- ☐ SVML
- ☐ Other

Categories

- ☐ Application-Targeted
- ☐ Arithmetic
- ☐ Bit Manipulation
- ☐ Cast
- ☐ Compare
- ☐ Convert
- ☐ Cryptography
- ☐ Elementary Math Functions
- ☐ General Support
- ☐ Load
- ☐ Logical
- ☐ Mask
- ☐ Miscellaneous
- ☐ Move
- ☐ OS-Targeted
- ☐ Probability/Statistics
- ☐ Random
- ☐ Set
- ☐ Shift
- ☐ Special Math Functions
- ☐ Store
- ☐ String Compare
- ☐ Swizzle
- ☐ Trigonometry
- ☐ Unknown

Release Notes

Download: Offline Intel® Intrinsics Guide

Additional resources:

- Intel® C++ Compiler Classic Developer Guide and Reference
- Intel® C++ Compiler community board

All throughput and latency data is sourced from Intel® 64 and IA-32 Architectures Software Developer Manuals .

Search Intel Intrinsics

void _mm_2intersect_epi32 (__m128i a, __m128i b, __mmask8* k1, __mmask8* k2)	vp2intersectd
void _mm256_2intersect_epi32 (__m256i a, __m256i b, __mmask8* k1, __mmask8* k2)	vp2intersectd
void _mm512_2intersect_epi32 (__m512i a, __m512i b, __mmask16* k1, __mmask16* k2)	vp2intersectd
void _mm_2intersect_epi64 (__m128i a, __m128i b, __mmask8* k1, __mmask8* k2)	vp2intersectq
void _mm256_2intersect_epi64 (__m256i a, __m256i b, __mmask8* k1, __mmask8* k2)	vp2intersectq
void _mm512_2intersect_epi64 (__m512i a, __m512i b, __mmask8* k1, __mmask8* k2)	vp2intersectq
void _aadd_i32 (int* __A, int __B)	aadd
void _aadd_i64 (__int64* __A, __int64 __B)	aadd
void _aand_i32 (int* __A, int __B)	aand
void _aand_i64 (__int64* __A, __int64 __B)	aand
__m128i _mm_abs_epi16 (__m128i a)	pabsw
__m128i _mm_mask_abs_epi16 (__m128i src, __mmask8 k, __m128i a)	vpabsw
__m128i _mm_maskz_abs_epi16 (__mmask8 k, __m128i a)	vpabsw
__m256i _mm256_abs_epi16 (__m256i a)	vpabsw
__m256i _mm256_mask_abs_epi16 (__m256i src, __mmask16 k, __m256i a)	vpabsw
__m256i _mm256_maskz_abs_epi16 (__mmask16 k, __m256i a)	vpabsw
__m512i _mm512_abs_epi16 (__m512i a)	vpabsw
__m512i _mm512_mask_abs_epi16 (__m512i src, __mmask32 k, __m512i a)	vpabsw
__m512i _mm512_maskz_abs_epi16 (__mmask32 k, __m512i a)	vpabsw
__m128i _mm_abs_epi32 (__m128i a)	pabsd
__m128i _mm_mask_abs_epi32 (__m128i src, __mmask8 k, __m128i a)	vpabsd
__m128i _mm_maskz_abs_epi32 (__mmask8 k, __m128i a)	vpabsd
__m256i _mm256_abs_epi32 (__m256i a)	vpabsd
__m256i _mm256_mask_abs_epi32 (__m256i src, __mmask8 k, __m256i a)	vpabsd
__m256i _mm256_maskz_abs_epi32 (__mmask8 k, __m256i a)	vpabsd
__m512i _mm512_abs_epi32 (__m512i a)	vpabsd
__m512i _mm512_mask_abs_epi32 (__m512i src, __mmask16 k, __m512i a)	vpabsd
__m512i _mm512_maskz_abs_epi32 (__mmask16 k, __m512i a)	vpabsd
__m128i _mm_abs_epi64 (__m128i a)	vpabsq

__m128i _mm_mask_abs_epi64 (__m128i src, __mmask8 k, __m128i a)	vpabsq
__m128i _mm_maskz_abs_epi64 (__mmask8 k, __m128i a)	vpabsq
__m256i _mm256_abs_epi64 (__m256i a)	vpabsq
__m256i _mm256_mask_abs_epi64 (__m256i src, __mmask8 k, __m256i a)	vpabsq
__m256i _mm256_maskz_abs_epi64 (__mmask8 k, __m256i a)	vpabsq
__m512i _mm512_abs_epi64 (__m512i a)	vpabsq
__m512i _mm512_mask_abs_epi64 (__m512i src, __mmask8 k, __m512i a)	vpabsq
__m512i _mm512_maskz_abs_epi64 (__mmask8 k, __m512i a)	vpabsq
__m128i _mm_abs_epi8 (__m128i a)	pabsb
__m128i _mm_mask_abs_epi8 (__m128i src, __mmask16 k, __m128i a)	vpabsb
__m128i _mm_maskz_abs_epi8 (__mmask16 k, __m128i a)	vpabsb
__m256i _mm256_abs_epi8 (__m256i a)	vpabsb
__m256i _mm256_mask_abs_epi8 (__m256i src, __mmask32 k, __m256i a)	vpabsb
__m256i _mm256_maskz_abs_epi8 (__mmask32 k, __m256i a)	vpabsb
__m512i _mm512_abs_epi8 (__m512i a)	vpabsb
__m512i _mm512_mask_abs_epi8 (__m512i src, __mmask64 k, __m512i a)	vpabsb
__m512i _mm512_maskz_abs_epi8 (__mmask64 k, __m512i a)	vpabsb
__m512d _mm512_abs_pd (__m512d v2)	vpandq
__m512d _mm512_mask_abs_pd (__m512d src, __mmask8 k, __m512d v2)	vpandq
__m128h _mm_abs_ph (__m128h v2)	...
__m256h _mm256_abs_ph (__m256h v2)	...
__m512h _mm512_abs_ph (__m512h v2)	...
__m64 _mm_abs_pi16 (__m64 a)	pabsw
__m64 _mm_abs_pi32 (__m64 a)	pabsd
__m64 _mm_abs_pi8 (__m64 a)	pabsb
__m512 _mm512_abs_ps (__m512 v2)	vpandd
__m512 _mm512_mask_abs_ps (__m512 src, __mmask16 k, __m512 v2)	vpandd
__m128d _mm_acos_pd (__m128d a)	...
__m256d _mm256_acos_pd (__m256d a)	...
__m512d _mm512_acos_pd (__m512d a)	...
__m512d _mm512_mask_acos_pd (__m512d src, __mmask8 k, __m512d a)	...
__m128h _mm_acos_ph (__m128h a)	...
__m256h _mm256_acos_ph (__m256h a)	...
__m512h _mm512_acos_ph (__m512h a)	...
__m512h _mm512_mask_acos_ph (__m512h src, __mmask32 k, __m512h a)	...
__m128 _mm_acos_ps (__m128 a)	...
__m256 _mm256_acos_ps (__m256 a)	...
__m512 _mm512_acos_ps (__m512 a)	...
__m512 _mm512_mask_acos_ps (__m512 src, __mmask16 k, __m512 a)	...
__m128d _mm_acosh_pd (__m128d a)	...
__m256d _mm256_acosh_pd (__m256d a)	...
__m512d _mm512_acosh_pd (__m512d a)	...
__m512d _mm512_mask_acosh_pd (__m512d src, __mmask8 k, __m512d a)	...
__m128h _mm_acosh_ph (__m128h a)	...
__m256h _mm256_acosh_ph (__m256h a)	...
__m512h _mm512_acosh_ph (__m512h a)	...

`__m512h _mm512_mask_acosh_ph (__m512h src, __mmask32 k, __m512h a)`	...
`__m128 _mm_acosh_ps (__m128 a)`	...
`__m256 _mm256_acosh_ps (__m256 a)`	...
`__m512 _mm512_acosh_ps (__m512 a)`	...
`__m512 _mm512_mask_acosh_ps (__m512 src, __mmask16 k, __m512 a)`	...
`__m128i _mm_add_epi16 (__m128i a, __m128i b)`	paddw
`__m128i _mm_mask_add_epi16 (__m128i src, __mmask8 k, __m128i a, __m128i b)`	vpaddw
`__m128i _mm_maskz_add_epi16 (__mmask8 k, __m128i a, __m128i b)`	vpaddw
`__m256i _mm256_add_epi16 (__m256i a, __m256i b)`	vpaddw
`__m256i _mm256_mask_add_epi16 (__m256i src, __mmask16 k, __m256i a, __m256i b)`	vpaddw
`__m256i _mm256_maskz_add_epi16 (__mmask16 k, __m256i a, __m256i b)`	vpaddw
`__m512i _mm512_add_epi16 (__m512i a, __m512i b)`	vpaddw